A MAJOR PROJECT PHASE 1 REPORT

ON

"AI-AGENT-MANAGED HACKATHON PRIZE ESCROW PLATFORM"

Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

SUBJECT CODE - MVJ22CDP65

Submitted By

1MJ23CD038	**PAVAN RAAJ M**
1MJ24CD413	**VIJAY KUMAR M**
1MJ23CD045	**SHASHANK VA**
1MJ23CD024	**KARTHIK M**

Under the Guidance of

Mr. Kushal A

Assistant Professor, Department of CSE (Data Science)

COLLEGE OF ENGINEERING
Since 1982
Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC)

MVJ COLLEGE OF ENGINEERING

Near ITPB, Whitefield, Bangalore – 560067

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

CERTIFICATE

This is to certify that the Major project work titled "AI-AGENT-MANAGED HACKATHON PRIZE ESCROW PLATFORM" is carried out by us, who are the Bonafide students of MVJ College of Engineering, Bengaluru, in partial fulfillment for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering (DS) of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/ suggestions indicated for the Internal Assessment have been incorporated in our major project report deposited in the department library. The Major project report has been approved as it satisfies the academic requirements with respect to the prescribed project work by the institution for the said degree.

Signature of Guide
Mr. Kushal A
Assistant Professor
Department of CSE (DS)

Signature of HOD
Prof. B Sivasakthi
Department of CSE (DS)

EXTERNAL EXAMINERS

Name of examiners:

Signature with date

MVJ COLLEGE OF ENGINEERING

Near ITPB, Whitefield, Bangalore – 560067

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

DECLARATION

We, students of Sixth semester B.E., Department of Computer science and Engineering (DS), MVJ College of Engineering, Bengaluru, hereby declare that the Major project titled **"AI BASED AUTOMATED BUSINESS REQUIREMENT GENERATION AGENT "** has been carried out by us and submitted in partial fulfilment for the award of Degree of Bachelor of Engineering in Computer science and Engineering (Data Science) during the year 2025 - 2026. Further, we declare that the content of the dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any Intellectual Property Rights generated out of this project carried out at MVJCE will be the property of MVJ College of Engineering, Bengaluru and we will be one of the authors of the same.

1MJ23CD019	KARTHIK M
1MJ23CD028	MONISH K S
1MJ23CD037	PASUPULETI PUNITH
1MJ23CD043	SAI MAHANTH L N

Place: BENGALURU Date:

ABSTRACT

Requirements Engineering (RE) is a critical phase in the software development lifecycle, responsible for capturing, analyzing, and documenting stakeholder needs. However, in modern agile and distributed environments, requirements are often scattered across unstructured communication sources such as emails, meeting transcripts, chat logs, and collaborative platforms. This fragmentation makes the process of creating Business Requirement Documents (BRDs) manual, time-consuming, and highly prone to inconsistencies, omissions, and requirement drift. Business analysts typically spend a significant portion of their time consolidating and interpreting such data, which reduces productivity and delays project timelines.

This project proposes an **AI-powered system for automated Business Requirement Document (BRD) generation** using advanced Natural Language Processing (NLP) techniques, Retrieval-Augmented Generation (RAG), and a multi-agent orchestration framework. The system is designed to ingest unstructured textual data from multiple sources, preprocess and filter irrelevant content, and extract meaningful functional and non-functional requirements. By leveraging semantic similarity and vector embeddings, relevant information is retrieved from a knowledge base and provided as grounded context to large language models (LLMs), thereby reducing hallucination and improving the factual accuracy of generated outputs.

The core of the system is a **multi-agent architecture**, where specialized AI agents collaborate to perform distinct tasks. The Analyst Agent identifies stakeholder requirements and system functionalities, the Architect Agent focuses on technical constraints and non-functional requirements, and the Critic Agent validates consistency, completeness, and logical correctness of the generated document. This collaborative approach enhances the quality and reliability of the final BRD.

The system is implemented using modern AI tools and frameworks, including LLM APIs for text generation, vector databases for efficient retrieval, and backend frameworks for seamless integration. The use of high-performance inference infrastructure enables near real-time document synthesis, making the system suitable for practical deployment in dynamic project environments. Additionally, the system maintains **traceability by linking each generated requirement to its original source**, ensuring transparency and auditability.

Experimental evaluation demonstrates that the proposed system significantly reduces manual effort, improves requirement coverage, and minimizes inconsistencies compared to traditional approaches. By automating the BRD generation process, the system enhances efficiency, supports better decision-making, and provides a scalable solution for modern software development practices.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany a successful completion of any task would be incomplete without the mention of people who made it possible. Success is the epitome of hard work and perseverance, but steadfast of all is encouraging guidance.

So, with gratitude we acknowledge all those whose guidance and encouragement served as beacon of light and crowned our effort with success.

We are thankful to **Dr. Ajayan K R, Principal of MVJCE** for his encouragement and support throughout the project work.

We are thankful to **Dr. Salim A, Dean, School of CSE, MVJCE** for his encouragement and support throughout the project work.

We are thankful to **Dr. Kumar R, Controller of Examinations,** for his encouragement and support throughout the project work.

We are also thankful to Prof. B Sivasakthi**, HOD, CSE (DS) Department** for her incessant encouragement and all the help during the project work.

We consider it a privilege and honor to express our sincere gratitude to our guide **Ms. Jija J S, Assistant Professor, CSE (DS) Department** for her valuable guidance throughout the tenure of this project work, and whose support and encouragement made this work possible.

It is also an immense pleasure to express our deepest gratitude to all faculty members of our department for their cooperation and constructive criticism offered, which helped us a lot during our project work.

Finally, we would like to thank all our family members and friends whose encouragement and support was invaluable.

Thanking You

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

In the modern software development landscape, Requirements Engineering (RE) plays a vital role in ensuring the successful design and implementation of software systems. It involves the systematic process of gathering, analyzing, documenting, and validating the needs and expectations of stakeholders. A well-defined set of requirements serves as the foundation for all subsequent stages of development. However, with the rapid adoption of agile methodologies and distributed team environments, requirement-related information is no longer confined to structured documents but is dispersed across unstructured communication channels such as emails, meeting transcripts, chat conversations, and collaborative tools.

This shift has introduced significant challenges in the requirements engineering process. Business analysts must manually collect, interpret, and consolidate information from multiple sources, which is time-consuming and error-prone. Important details may be overlooked or inconsistently documented, leading to issues such as ambiguity, redundancy, and requirement drift. These challenges negatively impact software quality, increase development costs, and delay project timelines. Existing approaches rely heavily on manual effort or basic Natural Language Processing (NLP) techniques, which are insufficient for handling complex, context-rich data and lack proper traceability.

Recent advancements in Artificial Intelligence, particularly in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG), have enabled more effective processing of unstructured data. These technologies allow systems to understand natural language, extract relevant information, and generate coherent outputs. Additionally, multi-agent systems introduce task specialization, where multiple AI agents collaborate to perform different stages of requirement analysis, improving overall accuracy and consistency.

In this context, the proposed project presents an **AI-powered system for automated Business Requirement Document (BRD) generation**. The system leverages NLP techniques, RAG architecture, and a multi-agent framework to extract relevant information from unstructured data sources, classify requirements, and generate structured BRDs in a standardized format. By incorporating semantic filtering, knowledge retrieval, and agent-based validation, the system ensures high-quality output with improved accuracy and traceability. The primary objective is to reduce manual effort, enhance consistency, and provide a scalable solution for modern software development environments.

CHAPTER 2

LITERATURE SURVEY

2.1 iReDev: A Knowledge-Driven Multi-Agent Framework for Intelligent Requirements Development

AUTHORS: Ataei et al

PUBLISHED ON: 2025

This paper presents iReDev, a knowledge-driven multi-agent framework designed for structured and intelligent requirements development. The system employs multiple specialized AI agents that leverage a large domain knowledge base to analyze, validate, and structure software requirements. Each agent is assigned a distinct functional role — such as requirement extraction, conflict resolution, and structured formatting — enabling a collaborative pipeline that mirrors the workflow of a human requirements engineering team. The framework improves the quality, completeness, and consistency of requirements through tight agent coordination and knowledge-guided reasoning. However, it faces challenges in managing complex inter-agent dependencies and requires an extensive, well-curated domain-specific knowledge base to operate effectively. Future work suggests integrating the framework with adaptive requirement engineering systems to improve scalability and reduce setup overhead.

2.2 Bridging Humans and LLMs: Investigating Human-AI Collaboration in Multi-agent Requirements Analysis for Organizational AI Adoption

AUTHORS: Abdul Sami, Zheying Zhang, Muhammad Waseem et al

PUBLISHED ON: 2026

This study investigates whether a multi-agent system built on LLMs, supported by structured human input, can meaningfully assist in requirements analysis for organizational AI adoption planning. Using a mixed-method approach involving case studies with four companies and feedback from nine participants, the research evaluates both the relevance and goal-alignment of AI-generated requirements. Results indicate that iterative human feedback played a critical role in improving the completeness and feasibility of generated requirements, often converging within just two rounds of refinement. Participants found the system useful in identifying requirements that aligned with strategic organizational goals, while also noting that human involvement was

essential for validating technical details and contextual accuracy. Key areas flagged for improvement include usability, transparency of the AI's reasoning process, and scalability for broader organizational deployment. The study concludes that LLM-based multi-agent systems hold strong potential for strategic AI planning when designed with meaningful human-in-the-loop mechanisms and calls for future work to improve system explainability and stakeholder engagement features.

2.3 Auto-scaling LLM-based Multi-agent Systems through Dynamic Integration of Agents

AUTHORS: Ravindu Perera, Anuradha Basnayake, Manjusri Wickramasinghe

PUBLISHED ON: 2025

This paper addresses one of the key limitations of existing LLM-based multi-agent systems — their static, predefined agent architectures that restrict adaptability in dynamic environments. The authors propose two novel one-shot approaches: Initial Automatic Agent Generation (IAAG), which automatically sets up an agent pool based on the user's initial context, and Dynamic Real-Time Agent Generation (DRTAG), which creates and integrates new agents on the fly as the conversation evolves. Both approaches utilize advanced prompt engineering techniques, including persona pattern prompting, chain prompting, and few-shot prompting, to enable one existing agent to generate another agent with an appropriate role and system prompt. Evaluations conducted in a simulated hospital environment demonstrated that DRTAG significantly outperformed static Autogen-based architectures across all measured dimensions — including task-related content coverage, lexical diversity, keyword richness, and thematic relevance. The study highlights that dynamically scaling the number of agents in a system leads to richer, more contextually comprehensive multi-agent conversations. These findings pave the way for building truly adaptive, intelligent AI systems capable of handling complex, evolving real-world tasks without requiring extensive manual configuration.

2.4 TraceLLM: Leveraging Large Language Models with Prompt Engineering for Enhanced Requirements Traceability

AUTHORS: Alturayeif et al

PUBLISHED ON: 2026

This paper proposes TraceLLM, a system that applies carefully designed prompt engineering techniques with LLMs to create and maintain traceability links between software requirements and other project artifacts such as design documents, test cases, and implementation code. Traceability is a critical but often neglected aspect of requirements engineering, as losing sight of where a requirement originated can lead to inconsistencies and audit failures later in the development of lifecycle. TraceLLM addresses this by automatically generating and managing trace links, improving the explainability and auditability of the overall requirement documentation process. The system ensures that every requirement in a generated document can be traced back to the specific source — whether an email, meeting transcript, or user story — from which it was derived. While the approach significantly improves requirement transparency and accountability, its performance is highly sensitive to the quality and specificity of prompt design. Additionally, the system performs best on structured datasets and may struggle to generalize reliably across unstructured or domain-specific communication data without additional tuning.

2.5 RTADev: Intention Aligned Multi-Agent Framework for Software Development

AUTHORS: Jian Liu, Gang Wang et al

PUBLISHED ON: 2025

This paper presents RTADev, a multi-agent framework specifically designed to align AI agent behavior with the underlying intentions of software developers and stakeholders. Rather than simply processing surface-level instructions, RTADev focuses on understanding the deeper intent behind developer communication — such as informal Slack messages, code comments, and meeting notes — and translating it into structured, actionable requirement artifacts. The framework employs multiple coordinated agents to handle distinct phases of this pipeline, including intent interpretation, requirement formalization, and artifact generation, ensuring that the system remains aligned with human objectives throughout the process. Experimental evaluations show that the framework improves the accuracy and relevance of requirement extraction compared to single-agent baselines, particularly in scenarios involving ambiguous or informal communication. However, the system's effectiveness is dependent on the availability of large, annotated training datasets to calibrate the intent-alignment mechanisms. Limited real-world validation and the absence of domain-specific fine-tuning remain key challenges that future research will need to address for broader deployment.

2.6 Sustainable Digitalization of Business with Multi-Agent RAG and LLM

AUTHORS: Sahoo et al

PUBLISHED ON: 2025

This work presents a multi-agent architecture that combines Retrieval-Augmented Generation (RAG) with Large Language Models for enterprise documentation and business knowledge management. The key innovation is grounding LLM-generated outputs in a structured, up-to-date knowledge base, which significantly reduces the risk of hallucinated or factually incorrect content — a critical concern in professional documentation contexts. Multiple agents work collaboratively within the system, with some responsible for retrieving relevant knowledge chunks from the vector store and others handling reasoning, synthesis, and structured document generation. The system is particularly well-suited for scenarios involving large volumes of organizational data, such as policy documents, business process records, and project documentation. While the architecture demonstrates strong improvements in output accuracy and factual reliability, it requires the maintenance of a large, well-organized knowledge base and involves considerable system complexity. High infrastructure costs and the challenge of managing multi-agent coordination at scale are noted as practical limitations, with real-time business documentation and adaptive knowledge management identified as promising directions for future development.

2.7 Enhancing Technical Document Compliance Review through a Context-Aware Generative AI Framework

AUTHORS: Gulave et al

PUBLISHED ON: 2025

This study introduces a context-aware generative AI framework specifically designed to automate the compliance review of technical documents against predefined regulatory, organizational, or industry standards. The system leverages contextual understanding of document structure, content, and intent to identify gaps, inconsistencies, and non-compliant sections — tasks that traditionally require significant manual effort from domain experts. By integrating generative AI with contextual analysis modules, the framework not only flags potential compliance issues but also suggests corrective content, making it a semi-automated compliance assistant rather than just a checker. The system demonstrates measurable improvements in document review speed and consistency, reducing the subjectivity that often accompanies manual audits. However, the model's accuracy and generalizability are strongly tied to the diversity and quality of training data. Performance degradation is observed when applied to documents from domains significantly different from those used during training, indicating that domain adaptation and continual learning mechanisms would be necessary for broader enterprise adoption.

2.8 Leveraging RAG and LLMs for Access Control Policy Extraction from User Stories in Agile Software Development

AUTHORS: IEEE Research

PUBLISHED ON: 2025

This paper explores the use of a Retrieval-Augmented Generation (RAG) pipeline in combination with Large Language Models to automatically extract access control policies and security-related requirements from agile user stories. Security requirements are among the most overlooked aspects of early-stage requir ements engineering, especially in agile environments where speed is prioritized over documentation rigor. The proposed system addresses this gap by processing user stories through a RAG pipeline that retrieves relevant security policy templates and regulatory references, enabling the LLM to generate accurate, context-aware access control specifications. The approach is evaluated on a dataset of agile user stories and demonstrates strong performance in identifying security constraints that would otherwise be missed in standard requirements analysis. Key limitations include the requirement for well-structured and consistently formatted input data, and the system's limited ability to generalize across different application domains without retraining or significant prompt adjustment. Future research directions include automated security requirement generation integrated directly into agile project management workflows.

2.9 Generative Artificial Intelligence for Requirements Engineering in Software Development – Analysis of the State-of-the-Art

AUTHORS: Perera et al

PUBLISHED ON: 2026

This paper offers a comprehensive state-of-the-art analysis of how generative artificial intelligence is being applied to requirements engineering across the software development lifecycle. The study surveys a wide range of techniques and models — including GPT-based systems, transformer architectures, and hybrid AI pipelines — and evaluates their effectiveness in automating tasks such as requirement elicitation, specification, validation, and documentation. It highlights that generative AI has demonstrated considerable promise in reducing the time and effort required to produce formal requirement documents, particularly when dealing with large, complex, or rapidly evolving projects. The paper also candidly addresses the risks associated with AI-generated requirements, most notably the tendency of generative models to produce plausible-sounding but factually incorrect or incomplete outputs — commonly referred to as hallucinations. The authors

argue that robust validation mechanisms, human-in-the-loop review processes, and domain-specific fine-tuning are essential prerequisites for safely deploying generative AI in production-grade requirements engineering environments. The paper concludes by calling for greater standardization of evaluation benchmarks and better integration with enterprise project management tools as key future priorities.

2.10 Generative AI for Requirements Engineering: A Systematic Literature Review

AUTHORS: Haowei Cheng, Jati H. Husen, Yijun Lu et al

PUBLISHED ON: 2024

This systematic literature review provides one of the most comprehensive analyses of generative AI applications in requirements engineering to date, examining 105 articles published between 2019 and 2024 from major academic databases including Scopus, IEEE Xplore, and arXiv. The review reveals that GPT series models dominate current applications, appearing in 67.3% of studied papers, while alternative architectures such as Claude, PaLM, and open-source models like LLaMA account for only a small fraction of implementations. Among the prompt engineering strategies analyzed, few-shot learning emerged as the most widely adopted approach at 40.2%, followed by zero-shot learning at 36.8%, with instruction-based prompts being the dominant format at 68.4%. The review identifies interpretability (61.9%), reproducibility (52.4%), and controllability (47.6%) as the three most critical unresolved challenges in the field. A notable finding is the concentration of research in requirements analysis and elicitation phases, while requirements management remains significantly underexplored at just 5.4% of studies. The authors call for more diverse model exploration, standardized evaluation frameworks, and research into real-world industrial deployment of generative AI for requirements engineering

2.11 AI-Based Multiagent Approach for Requirements Elicitation and Analysis

AUTHORS: Malik Abdul Sami et al

PUBLISHED ON: 2024

This paper, which serves as the foundational reference for the proposed project, introduces an LLM-based multi-agent system that automates the two most labor-intensive phases of requirements engineering: elicitation and analysis. Rather than relying on a single monolithic AI model, the system distributes responsibility across multiple specialized agents — each handling a specific sub-task such as extracting stakeholder needs, detecting requirement conflicts, or

generating formal requirement specifications. The agents communicate iteratively, refining outputs based on each other's findings to produce structured, high-quality requirement artifacts that closely match human expert standards. The paper demonstrates that multi-agent collaboration enables more nuanced understanding of complex requirement scenarios compared to single-agent baselines, and that the system can be applied to diverse project types with minimal domain-specific configuration. Key limitations include the high computational cost of running large multi-agent pipelines and the current lack of deep integration with real-world communication platforms such as email or messaging systems. The proposed project directly builds this work by extending the multi-agent paradigm to automate the full BRD generation pipeline from raw, multi-source communication data.

2.12 From Learning Agents to Agile Software: Reinforcement Learning's Transformative Role in Requirements Engineering

AUTHORS: Siddiqui et al

PUBLISHED ON: 2024

This paper explores the novel application of reinforcement learning (RL) agents to the domain of requirements engineering, with a particular focus on dynamic requirement prioritization in agile software development environments. Unlike static prioritization methods that rank requirements based on fixed heuristics, the RL-based approach allows agents to learn and adapt their prioritization strategies over time based on feedback signals derived from project outcomes, stakeholder satisfaction scores, and development velocity metrics. The framework demonstrates the ability to handle changing project conditions gracefully — reprioritizing requirements in real time as new information emerges — which is especially valuable in fast-paced agile settings where requirements frequently evolve. Experimental results show that RL-guided prioritization outperforms traditional methods such as MoSCoW and weighted scoring in dynamic scenarios with frequently shifting stakeholder priorities. However, the approach poses significant challenges for small or early-stage projects where there is insufficient historical data to meaningfully train the RL agents. Designing effective reward functions that accurately reflect real-world project success criteria also remains a non-trivial engineering challenge, and the authors suggest that hybrid approaches combining RL with rule-based heuristics may offer a more practical solution for near-term deployment.

CHAPTER 3

PROBLEM STATEMENT

In modern software development environments, a significant portion of requirement-related information is generated through unstructured communication channels such as emails, meeting transcripts, chat logs, and collaborative tools. These sources contain valuable insights about stakeholder expectations, system functionalities, and constraints. However, due to their unstructured and scattered nature, extracting meaningful and actionable requirements from such data remains a major challenge. The lack of a centralized and structured approach to requirement collection leads to inefficiencies in the Requirements Engineering (RE) process.

Currently, Business Analysts are responsible for manually reviewing and consolidating information from multiple communication sources to create Business Requirement Documents (BRDs). This process is highly time-consuming, repetitive, and prone to human errors. Important requirements may be missed, duplicated, or misinterpreted, resulting in inconsistencies and ambiguities in the final documentation. These issues often lead to requirement drift, where the implemented system deviates from the original stakeholder expectations, ultimately affecting software quality and project success.

Although recent advancements in Artificial Intelligence, particularly Large Language Models (LLMs), have shown promise in automating text understanding and generation, they are often susceptible to issues such as hallucination and lack of grounding in factual data. Without proper mechanisms to ensure contextual relevance and accuracy, these models may generate incorrect or unsupported requirements, limiting their practical applicability in real-world scenarios.

Therefore, there is a clear need for an intelligent and automated system that can efficiently process unstructured communication data, accurately extract both functional and non-functional requirements, and generate structured Business Requirement Documents. Such a system should minimize human intervention, reduce errors, maintain consistency, and ensure traceability between requirements and their original sources. Additionally, it should incorporate mechanisms to improve the reliability and accuracy of AI-generated outputs, making it suitable for practical use in modern software development environments.

CHAPTER 4

EXISTING SYSTEM AND PROPOSED SYSTEM

4.1 EXISTING SYSTEM

In current software development practices, the process of creating Business Requirement Documents (BRDs) is largely manual and relies heavily on human effort. Requirements are typically gathered from various unstructured communication sources such as emails, meeting notes, chat conversations, and stakeholder discussions. Business Analysts are responsible for reviewing these sources, identifying relevant information, and compiling it into structured documents.

Traditional approaches to requirement engineering use basic documentation tools such as word processors, spreadsheets, and requirement management systems like Jira or Confluence. While these tools assist in organizing and storing requirements, they do not automate the extraction or synthesis of information from raw data. Analysts must manually interpret the context, resolve ambiguities, and ensure consistency across different inputs.

Some existing systems incorporate basic Natural Language Processing (NLP) techniques to assist in requirement extraction. These systems may use keyword -based methods or simple machine learning models to identify important phrases or classify text. However, such approaches are limited in their ability to understand complex language, contextual relationships, and implicit requirements.

Recent advancements have introduced Large Language Models (LLMs) that can generate text and assist in documentation. However, when used independently, these models often lack grounding in actual data sources and may produce inaccurate or fabricated information (hallucinations). Additionally, most existing tools do not provide traceability between generated requirements and their original sources, making validation difficult.

Overall, existing systems either rely heavily on manual effort or provide limited automation, making the process inefficient, error-prone, and unsuitable for handling large volumes of unstructured data.

4.1.1 DISADVANTAGES OF EXISTING SYSTEM

- **Manual and Time-Consuming Process:** Requirement extraction and BRD creation require significant human effort, leading to reduced productivity and delays.
- **Prone to Human Error:** Important requirements may be missed, misinterpreted, or duplicated, resulting in inconsistencies and ambiguity in documentation.

- **Lack of Automation:** Existing tools do not effectively automate the extraction and synthesis of requirements from unstructured data sources.
- **Limited Context Understanding:** Traditional NLP-based systems fail to capture complex relationships, implicit requirements, and contextual meaning.
- **No Traceability:** Most systems do not link requirements to their original sources, making validation and auditing difficult.
- **Scalability Issues:** Handling large volumes of data from multiple communication channels becomes inefficient and difficult to manage.
- **Fragmented Information Sources:** Requirements are scattered across multiple platforms, leading to incomplete or inconsistent documentation.
- **LLM Hallucination Risk:** Standalone AI models may generate incorrect or unsupported requirements due to lack of grounding.

4.2 PROPOSED SYSTEM

The proposed system is an **AI-powered platform for automated Business Requirement Document (BRD) generation** designed to overcome the limitations of traditional requirement engineering methods. Unlike existing approaches that rely heavily on manual effort, the system automates the process of extracting, analyzing, and structuring requirements from unstructured communication sources such as emails, meeting transcripts, and chat logs.

The system begins by ingesting raw textual data from multiple sources and performing preprocessing steps such as noise removal, normalization, and segmentation. A semantic filtering mechanism is applied to identify and retain only relevant requirement-related information while eliminating irrelevant or redundant content. This ensures that the system focuses on meaningful data for further processing.

To enhance accuracy and contextual understanding, the system utilizes a **Retrieval-Augmented Generation (RAG)** approach. The processed data is converted into vector embeddings and stored in a vector database, enabling efficient semantic retrieval. When generating requirements, the system retrieves the most relevant information based on similarity and provides it as context to a Large Language Model (LLM). This grounding mechanism significantly reduces hallucinations and improves the reliability of the generated output.

A key feature of the proposed system is its **multi-agent architecture**, where specialized AI agents collaborate to generate a high-quality BRD. The Analyst Agent extracts functional requirements and stakeholder needs, the Architect Agent identifies non-functional requirements and technical constraints, and the Critic Agent evaluates the output for consistency, completeness, and logical correctness. This collaborative workflow ensures better organization and validation of requirements.

The final output is a structured BRD that follows standard documentation formats such as IEEE 830. The system also maintains **traceability** by linking each requirement to its original source, enabling easy verification and validation. Overall, the proposed system provides an intelligent, automated, and scalable solution for requirement engineering in modern software development environments.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

- **Automation of Requirement Engineering:** Reduces manual effort by automatically extracting and generating requirements from unstructured data.
- **Improved Accuracy and Consistency:** The use of RAG ensures that generated requirements are grounded in actual data, minimizing errors and inconsistencies.
- **Context-Aware Understanding:** Advanced NLP and LLMs enable better interpretation of complex language and implicit requirements.
- **Multi-Agent Collaboration:** Specialized agents improve the quality, organization, and validation of the generated BRD.
- **Traceability:** Each requirement is linked to its original source, enhancing transparency and ease of verification.
- **Scalability:** The system efficiently handles large volumes of data from multiple communication channels.
- **Reduced Requirement Drift:** Accurate extraction and validation help ensure alignment with stakeholder expectations.
- **Standardized Output:** Generates BRDs in a structured and consistent format, improving readability and usability.

CHAPTER 5

METHODOLOGY

5.1 Overview

The proposed system follows a structured pipeline that transforms unstructured communication data into a well-defined Business Requirement Document (BRD). The methodology integrates data preprocessing, semantic filtering, vector-based retrieval, and multi-agent reasoning to ensure accurate and traceable requirement extraction. The system is designed to handle real-world inputs such as emails and meeting transcripts and convert them into structured outputs using advanced Artificial Intelligence techniques.

5.2 System Architecture

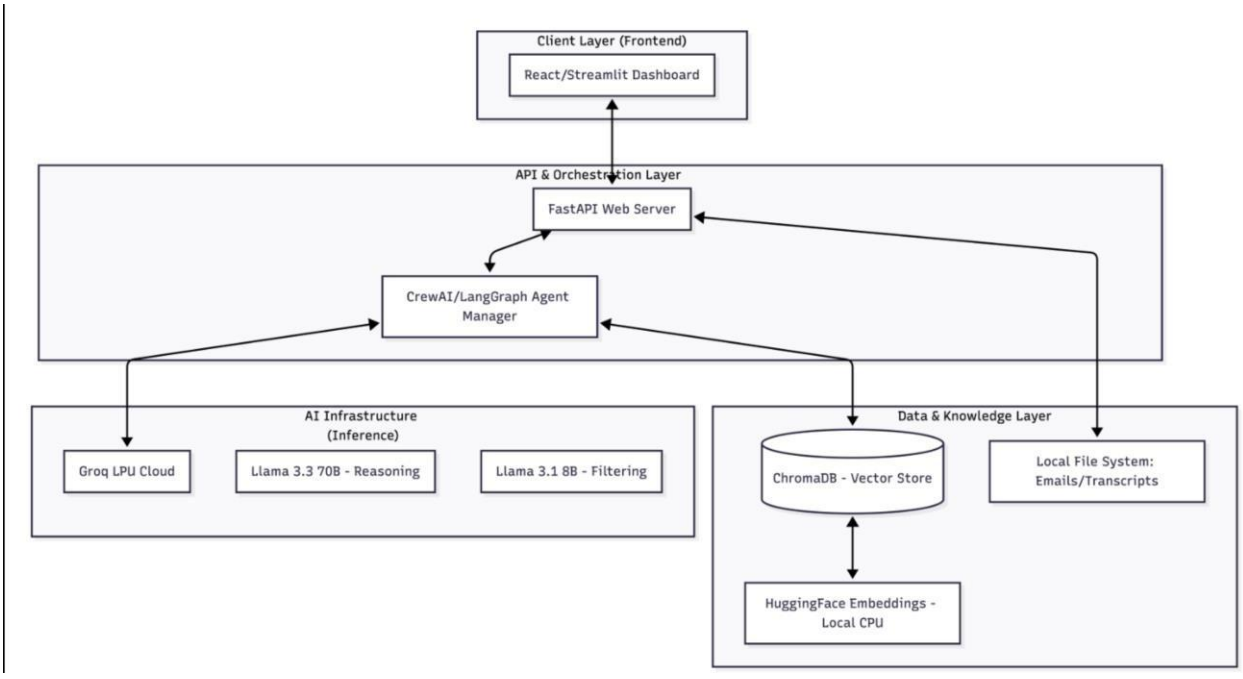

Figure 5.1: System Architecture

The overall architecture of the system is divided into four major layers:
Client Layer, API & Orchestration Layer, AI Infrastructure Layer, and Data & Knowledge Layer.

Client Layer (Frontend)

The client layer consists of a user interface built using React or Streamlit. It allows the user (typically a project manager or business analyst) to upload input data such as emails or transcripts and trigger BRD generation.

API & Orchestration Layer

This layer acts as the central control unit of the system. A FastAPI-based web server handles incoming requests and coordinates the execution of different components. It interacts with the **Agent Manager (CrewAI or LangGraph)**, which is responsible for managing the workflow of multiple AI agents.

AI Infrastructure Layer

This layer performs the core intelligence of the system. It utilizes high-performance inference through Groq LPU Cloud and multiple Large Language Models:

- A lightweight model (Llama 3.1 8B) is used for filtering and classification of relevant data.
- A powerful model (Llama 3.3 70B) is used for reasoning, synthesis, and BRD generation.

The multi-agent system operates within this layer, where different agents collaborate to extract, validate, and organize requirements.

Data & Knowledge Layer

This layer is responsible for storing and retrieving data efficiently:

- **ChromaDB** is used as a vector database to store embeddings of processed text.
- **HuggingFace Embeddings** generate vector representations of text.
- A local file system stores raw inputs such as emails and transcripts.

This layer enables semantic search and ensures that relevant information is retrieved during BRD generation.

5.3 System Workflow

The workflow of the system, as shown in the sequence diagram, describes the interaction between different components from input to output.

1. **User Input:**
   The user uploads emails or meeting transcripts through the interface.

2. **Data Preprocessing:**
   The data preprocessor cleans the input by removing noise such as signatures, advertisements, and irrelevant text.
3. **Embedding and Storage:**
   The cleaned data is converted into vector embeddings and stored in the vector database along with metadata.
4. **BRD Generation Request:**
   The user initiates a command to generate the BRD.
5. **Context Retrieval:**
   The system queries the vector database to retrieve the most relevant text chunks using semantic similarity.
6. **Multi-Agent Processing:**
   The retrieved context is passed to the multi-agent system:
   a. Analyst Agent extracts functional requirements.
   b. Architect Agent identifies non-functional requirements.
   c. Critic Agent validates and refines the output.
7. **LLM Processing:**
   The agents interact with the LLM (via Groq) to perform reasoning and generate structured BRD sections.
8. **Output Generation:**
   The final BRD is generated with proper formatting and traceability links to the original data sources.
9. **Result Display:**
   The generated document is returned to the user interface for viewing and further use.

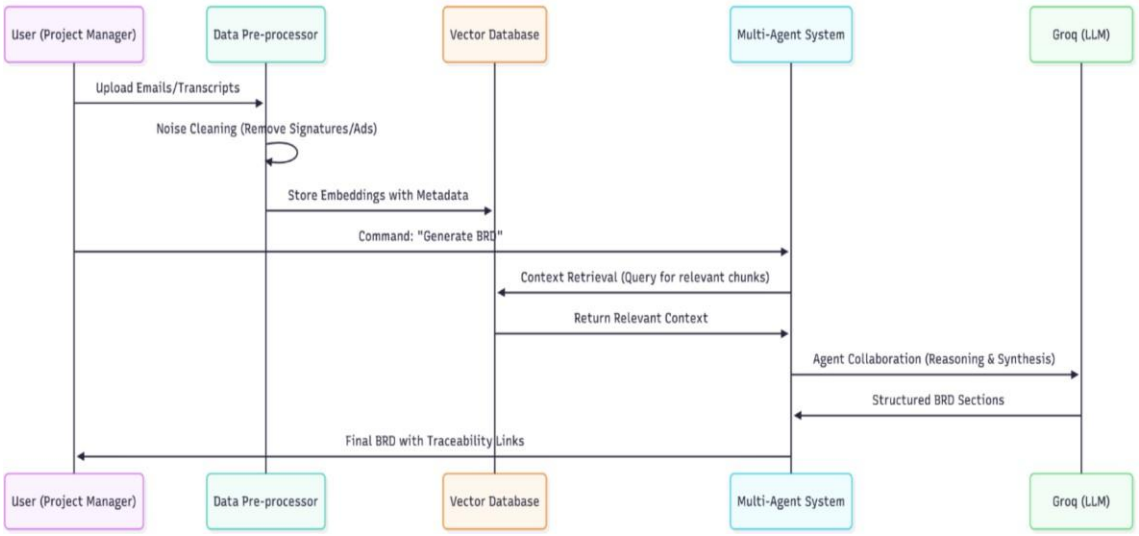

Fig 5.2: System Workflow

5.4 Detailed Methodology Steps

1. Data Acquisition

Input data is collected in the form of emails and meeting transcripts. These inputs represent real-world requirement discussions.

2. Data Preprocessing

The collected data is cleaned by removing noise such as HTML tags, signatures, timestamps, and irrelevant content. The text is normalized for consistent processing.

3. Semantic Filtering

A lightweight LLM is used to classify whether a text segment is relevant to requirements or not. This reduces noise and improves system efficiency.

4. Vectorization and Storage

The filtered text is converted into embeddings using sentence-transformer models and stored in ChromaDB along with metadata such as source and timestamp.

5. Retrieval-Augmented Generation (RAG)

When generating the BRD, the system retrieves the most relevant text segments based on semantic similarity. This ensures that the LLM operates on accurate and contextually relevant information.

6. Multi-Agent Collaboration

The system uses multiple AI agents with specialized roles:

- **Analyst Agent:** Extracts functional requirements and stakeholder needs.
- **Architect Agent:** Identifies non-functional requirements and constraints.
- **Critic Agent:** Ensures logical consistency and completeness.

7. BRD Generation

The LLM synthesizes the extracted information into a structured BRD format, including sections such as functional requirements, non-functional requirements, and constraints.

8. Traceability Mapping

Each generated requirement is linked back to its original source, ensuring transparency and easy validation.

CHAPTER 6

SYSTEM REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

The proposed system is designed to operate efficiently on standard computing hardware while leveraging cloud-based AI infrastructure for high-performance processing.

- **Processor:** Intel Core i5 (10th Gen or higher) or equivalent multi-core processor
- **RAM:** Minimum 8 GB (16 GB recommended for smooth execution of embedding and preprocessing tasks)
- **Storage:** Minimum 20 GB free disk space for storing datasets, embeddings, and application files
- **Network:** Stable high-speed internet connection for accessing LLM APIs (Groq Cloud)
- **GPU (Optional):** Not mandatory, as the system uses cloud-based inference, but can be used for local model experimentation

6.2 SOFTWARE REQUIREMENTS

The system is developed using modern software tools and frameworks for AI processing, backend orchestration, and frontend interaction.

Operating System

- Windows 10/11 or Linux (Ubuntu 20.04 or later)

Programming Language

- Python 3.10 or higher

Backend Framework

- FastAPI – for building RESTful APIs and handling system orchestration

Frontend Technologies

- Streamlit or React – for creating an interactive user interface

AI & Machine Learning Libraries

- LangChain / CrewAI / LangGraph – for multi-agent orchestration
- HuggingFace Transformers – for embeddings and NLP processing
- Sentence-Transformers – for generating vector embeddings

Vector Database

- ChromaDB – for storing and retrieving embeddings using semantic similarity

Large Language Model (LLM) API

- Groq Cloud (Llama 3.1 / Llama 3.3) – for high-speed inference and text generation

Data Processing Libraries

- Pandas – for data manipulation
- NumPy – for numerical operations
- NLTK / SpaCy – for text preprocessing

File Processing Tools

- Unstructured.io or similar libraries – for parsing emails and transcripts

Development Tools

- Visual Studio Code / PyCharm – for development and debugging
- Git & GitHub – for version control

CHAPTER 7

CONCLUSION

This project presented the design and implementation of an **AI-powered system for automated Business Requirement Document (BRD) generation**, addressing the key challenges associated with traditional requirements of engineering processes. In modern software development environments, requirement-related information is widely distributed across unstructured sources such as emails, meeting transcripts, and chat logs, making manual extraction and documentation inefficient and error-prone. The proposed system successfully demonstrates how advanced Artificial Intelligence techniques can be leveraged to automate and improve this process.

The system integrates Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and a multi-agent architecture to extract, analyze, and synthesize requirements from unstructured data. By incorporating semantic filtering and vector-based retrieval, the system ensures that only relevant and contextually accurate information is used during document generation. The use of RAG significantly reduces the problem of hallucination commonly associated with Large Language Models (LLMs), thereby improving the reliability of the generated output.

A key contribution of this project is the implementation of a **multi-agent system**, where specialized agents collaboratively perform different aspects of requirement engineering. The Analyst Agent focuses on identifying functional requirements and stakeholder needs, the Architect Agent captures non-functional requirements and system constraints, and the Critic Agent ensures consistency, completeness, and logical correctness. This collaborative approach enhances the overall quality of the generated BRD and reflects real-world requirement analysis workflows.

The system also emphasizes **traceability**, linking each generated requirement back to its original source. This feature improves transparency, enables easy validation, and increases user trust in the automated system. Furthermore, by utilizing high-performance inference through cloud-based infrastructure, the system achieves efficient and near real-time document generation, making it practical for real-world applications.

In conclusion, the proposed AI-based BRD generator offers a robust, efficient, and scalable solution for modern requirements engineering challenges. It bridges the gap between unstructured communication data and structured documentation, enabling organizat ions to streamline their development processes and improve decision-making. The project highlights the potential of combining RAG and multi-agent systems in real-world applications and lays the foundation for further advancements in intelligent software engineering tools.

REFERENCES

- Ataei et al., *"iReDev: A Knowledge-Driven Multi-Agent Framework for Intelligent Requirements Development,"* 2025.
- Abdul Sami, Zheying Zhang, Muhammad Waseem et al., *"Bridging Humans and LLMs: Investigating Human-AI Collaboration in Multi-agent Requirements Analysis for Organizational AI Adoption,"* 2026.
- Ravindu Perera, Anuradha Basnayake, Manjusri Wickramasinghe, *"Auto-scaling LLM-based Multi-agent Systems through Dynamic Integration of Agents,"* 2025.
- Alturayeif et al., *"TraceLLM: Leveraging Large Language Models with Prompt Engineering for Enhanced Requirements Traceability,"* 2026.
- Jian Liu, Gang Wang et al., *"RTADev: Intention Aligned Multi-Agent Framework for Software Development,"* 2025.
- Sahoo et al., *"Sustainable Digitalization of Business with Multi-Agent RAG and LLM,"* 2025.
- Gulave et al., *"Enhancing Technical Document Compliance Review through a Context-Aware Generative AI Framework,"* 2025.
- IEEE Research, *"Leveraging RAG and LLMs for Access Control Policy Extraction from User Stories in Agile Software Development,"* 2025.
- Perera et al., *"Generative Artificial Intelligence for Requirements Engineering in Software Development – Analysis of the State-of-the-Art,"* 2026.
- Haowei Cheng, Jati H. Husen, Yijun Lu et al., *"Generative AI for Requirements Engineering: A Systematic Literature Review,"* 2024.
- Malik Abdul Sami et al., *"AI-Based Multiagent Approach for Requirements Elicitation and Analysis,"* 2024.
- Siddiqui et al., *"From Learning Agents to Agile Software: Reinforcement Learning's Transformative Role in Requirements Engineering,"* 2024.